MODULE 4

Memory Management and Performance

Write efficient, reliable applications by managing memory wisely and optimizing performance.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MEMORY MANAGEMENT AND PERFORMANCE

Efficient memory management by the JVM ensures high performance, stability and scalability of Java applications.

MEMORY ALLOCATION

Object Creation

Objects are created in the Heap memory using new keyword.

Object Dereference

Objects with no references become eligible for Garbage Collection.

JVM MEMORY AREAS

HEAP (Shared)

- Young Generation
  - Eden Space
  - Survivor Spaces (S0, S1)
- Old Generation (Tenured)

NON-HEAP (Shared)

- **Metaspace** — Class metadata
- **Code Cache** — Compiled code
- **Thread Stacks** — Each thread has its stack
- **PC Registers** — Program counter for threads
- **Native Method Stack** — Native method execution

OBJECT FLOW

New Object → Young Generation (Eden) → Survivor Spaces (S0 / S1) → Old Generation (Tenured)

PERFORMANCE FACTORS

Garbage Collection

Efficient GC minimizes pause time and improves application responsiveness.

Memory Usage

Optimal memory usage reduces latency and improves throughput.

CPU Utilization

Balanced CPU usage ensures better execution and faster processing.

Tuning JVM

Proper JVM parameters (Xms, Xmx, GC type) boost performance.

Avoid Memory Leaks

Release unused objects and resources to maintain performance.

GARBAGE COLLECTION CYCLE

1. **Mark** — Identify live objects that are still reachable.
2. **Sweep** — Remove unreachable objects and free the memory.
3. **Compact** — Compact memory by eliminating fragmentation.
4. **Repeat** — Cycle repeats to maintain optimal memory.

BEST PRACTICES

- Use appropriate data structures
- Avoid memory leaks and unnecessary object creation
- Tune JVM heap size and GC options
- Monitor memory and GC using tools (jvisualvm, jmc)
- Prefer immutability and object pooling where suitable

MODULE OVERVIEW

Efficient use of resources for faster, scalable, and reliable applications.

FOCUS	OUTCOME	IMPACT	SKILL LEVEL	ESTIMATED TIME
Efficient Use of Resources	Faster, Scalable Applications	Reliable Systems at Scale	Intermediate	45 Minutes

This module covers how memory is allocated and managed in applications, common memory issues, and techniques to improve performance. You will learn to write code that is both efficient and scalable.

YOU WILL LEARN

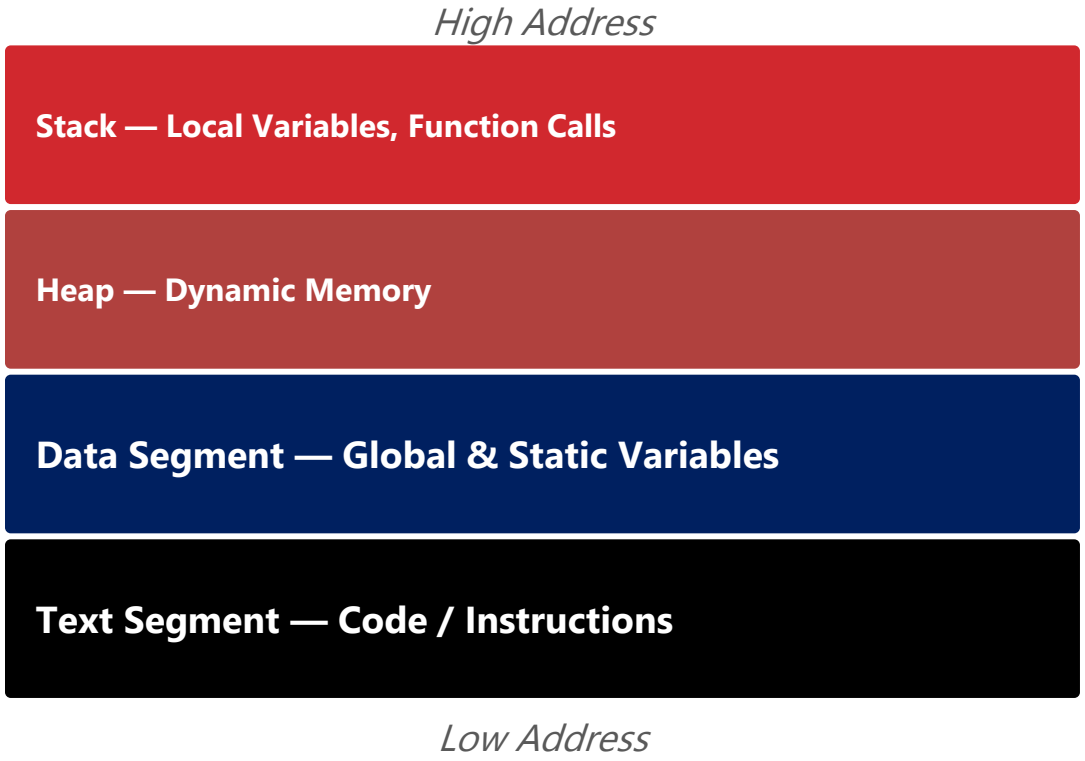
- How memory is allocated and deallocated
- Common memory management problems
- Tools and techniques to detect memory issues
- Performance metrics and profiling
- Best practices for writing performant code

KEY TAKEAWAY This module builds the foundation for writing memory-efficient, high-performance Java applications.

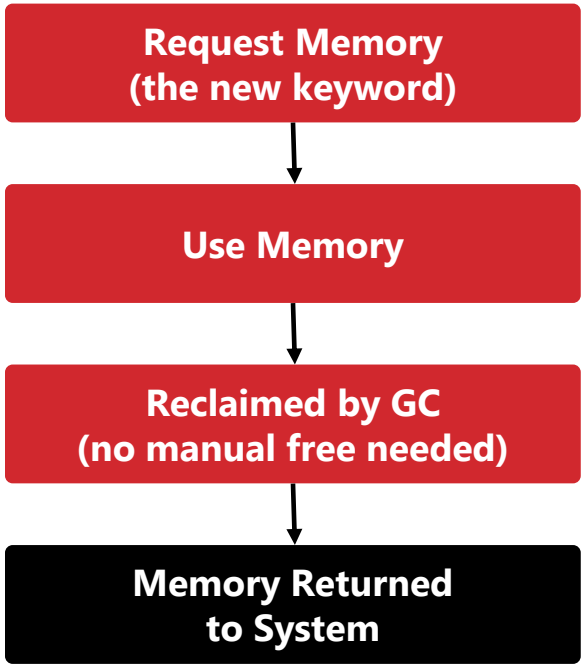
MEMORY MODEL OVERVIEW

How a running process organizes memory, and the lifecycle of a single allocation.

PROCESS MEMORY LAYOUT



MEMORY MANAGEMENT FLOW



KEY TAKEAWAY Every allocation follows the same lifecycle: request, use, release, and return to the system.

WHY MEMORY MANAGEMENT MATTERS

Memory is a finite resource — how you allocate, use, and release it directly shapes performance, reliability, cost, and code quality.

1. Better Performance

- Reduces CPU overhead (e.g., GC, paging)
- Faster response times and higher throughput

2. Higher Reliability

- Prevents leaks, overflows, and corruption
- Reduces crashes and unpredictable behavior

3. Scalability

- Handles more users and data
- Critical for cloud-native, distributed systems

4. Cost Efficiency

- Efficient apps do more with less hardware
- Saves money in cloud and on-prem environments

5. Better User Experience

- Fewer slowdowns and crashes
- Directly impacts adoption and retention

6. Foundation for Good Code

- Forces clean, thoughtful code
- Encourages the right data structures and lifetimes

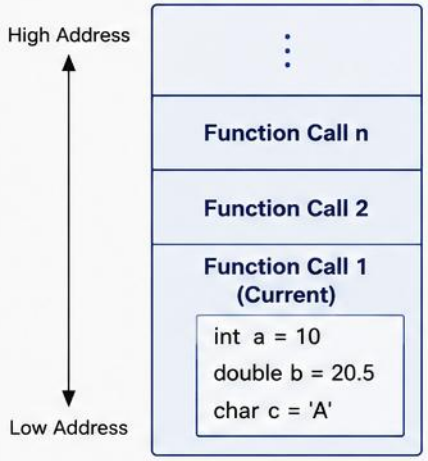
KEY TAKEAWAY Memory discipline is a business enabler, not just a technical detail.

STACK vs HEAP ALLOCATION

Stack and Heap are memory regions used by a program at runtime for different purposes.

STACK (Automatic Allocation)

Stack Memory



Characteristics

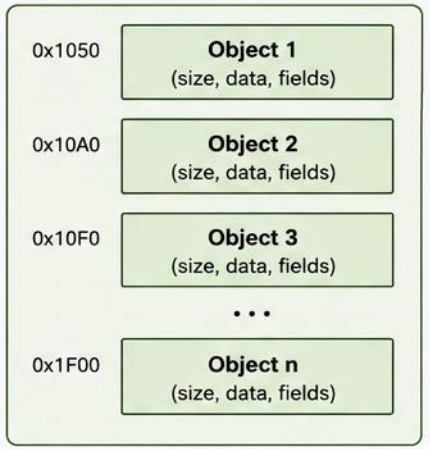
- Stores primitive data types and local variables.
- Memory is allocated and deallocated automatically.
- Follows LIFO (Last In First Out) order.
- Fast allocation and deallocation.
- Limited size.
- Each thread has its own stack.

Example (Java)

```
void method() {
    int a = 10;          // stored on Stack
    double b = 20.5;     // stored on Stack
    MyClass obj = new MyClass();    // reference on Stack
}                                   // object on Heap
```

HEAP (Dynamic Allocation)

Heap Memory



Characteristics

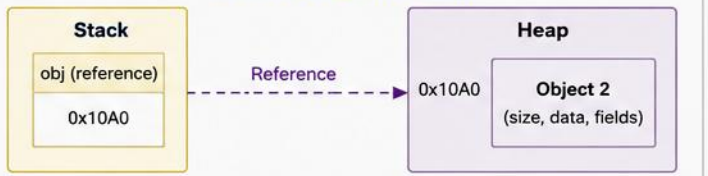
- Stores objects and dynamic data.
- Memory must be managed manually (or by Garbage Collector in Java).
- No specific order (not LIFO).
- Slower allocation and deallocation.
- Large size (limited by system memory).
- Shared among all threads.

Example (Java)

```
void method() {
    MyClass obj = new MyClass();  // obj reference on Stack
                                  // MyClass object on Heap
}
```

Aspect	Stack	Heap
Allocation	Automatic	Manual (or GC manages)
Deallocation	Automatic	Manual (or GC manages)
Speed	Faster	Slower
Size	Limited (MBs)	Large (GBs)
Access	Direct	Through reference
Used For	Primitive types, local variables, method calls	Objects, arrays, dynamic memory

Stack vs Heap in Action



💡 **Key Takeaway:** Stack is used for quick, temporary data. Heap is used for objects and data that need to live beyond the current scope.

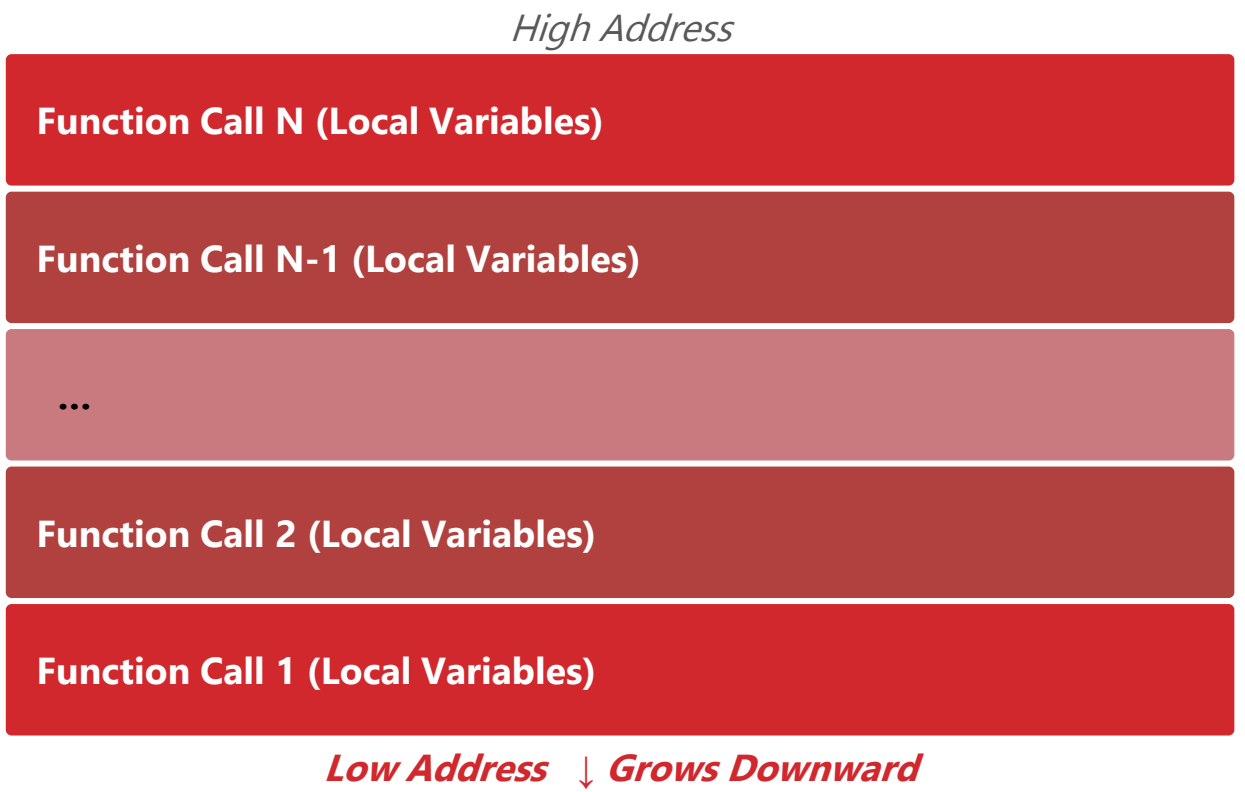
STACK (AUTOMATIC ALLOCATION)

Memory is managed automatically by the system, following LIFO order. (1 of 2)

HOW IT WORKS

- Allocated when a function is called.
- Deallocated when the function returns.
- Very fast allocation and deallocation.
- Size must be known at compile time.

PROPERTY	STACK
Speed	Very Fast
Management	Automatic
Size	Limited (KBs–MBs)
Lifetime	While function is active



KEY TAKEAWAY Each function call pushes a new frame onto the stack; returning pops it back off.

HEAP (DYNAMIC ALLOCATION)

Memory is allocated with new and reclaimed automatically by the garbage collector — more flexible but slower than the stack. (2 of 2)

HOW IT WORKS

- Allocated explicitly with the new keyword.
- Reclaimed automatically by the garbage collector (GC).
- Size can be determined at runtime.
- Risk of leaks if not managed properly.

PROPERTY	HEAP
`Speed`	Slower (overhead)
`Management`	Automatic (GC)
`Size`	Large (system memory)
`Lifetime`	Until unreachable, then GC'd

KEY TAKEAWAY Unlike the stack, heap blocks are allocated and freed in any order, which is what causes fragmentation.

TRY IT YOURSELF — EXERCISE 1: STACK VS HEAP BASICS

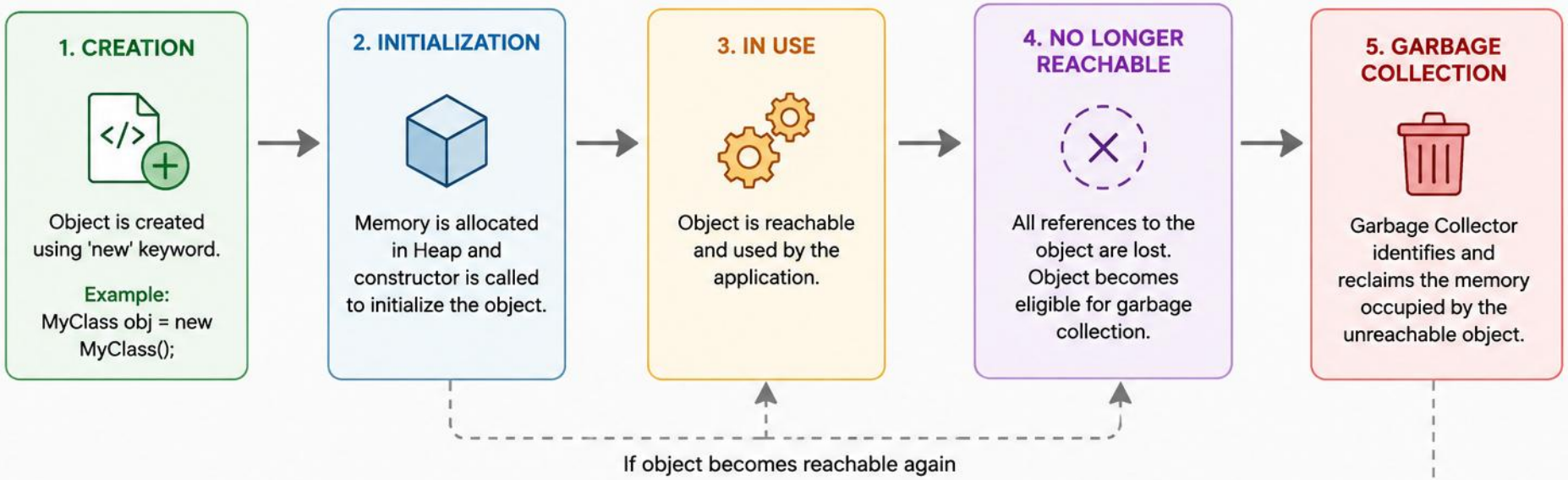
Reinforces: what lives on the stack vs. the heap, as just covered.

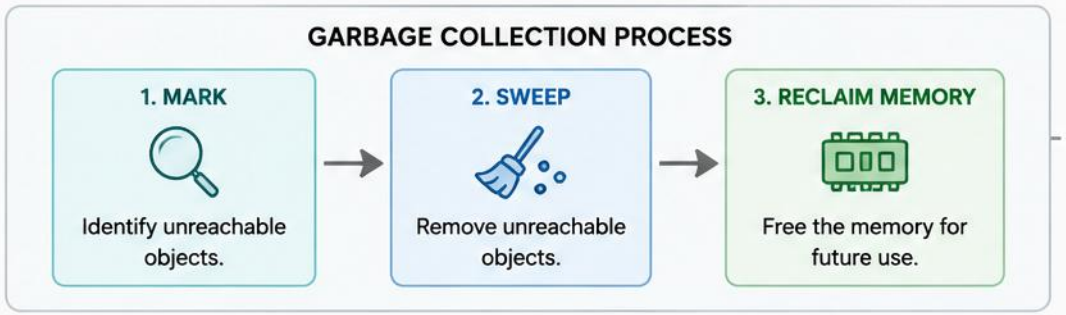
#	TITLE	FOCUS
1	Goal	Write a tiny program with local variables plus one new object; note what lives on stack vs. heap
2	Do this	Print the object's field, then write a 2-4 sentence observation
3	Key concept	Local variables and references live on the stack; the object itself lives on the heap
4	Reference	exercises/exercise-01-stack-vs-heap.md

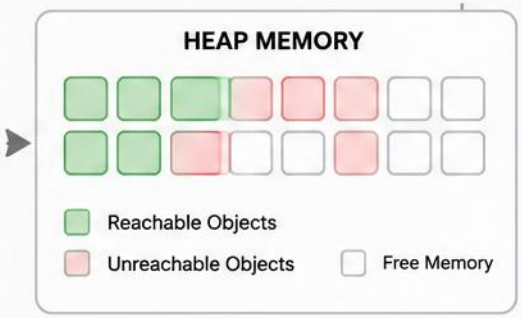
TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-stack-vs-heap.md.

OBJECT LIFECYCLE IN JAVA

The life of an object from creation to garbage collection







Key Points

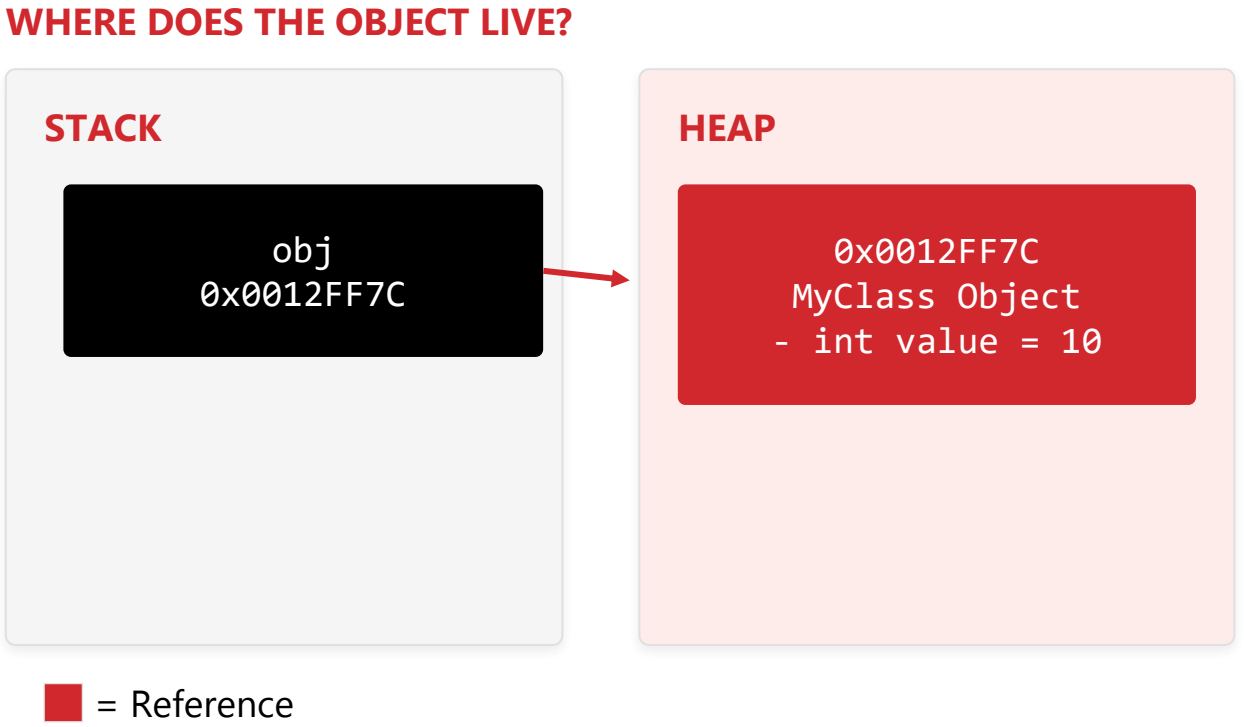
- All objects are created in Heap memory.
- An object is eligible for garbage collection only when it is unreachable.
- JVM's Garbage Collector automatically reclaims memory.

EXAMPLE FLOW: WHERE DOES THE OBJECT LIVE?

The full lifecycle in code, and the stack reference that points to it on the heap.

```
MyClass obj = new MyClass();  // 1. Creation
obj.setValue(10);             // 2. Init
obj.doWork();                 // 3. In Use
obj = null;                   // 4. No Longer
Ref'd
// eligible for GC              // 5. Eligible
// GC reclaims when it runs     // 6. Collected
```

System.gc() only suggests garbage collection — the JVM decides when it actually runs.
Unreachable does not mean immediately collected — the object may still sit in the heap until GC runs.



KEY TAKEAWAY The stack stores references (variables); the heap stores the actual objects they point to.

TRY IT YOURSELF — EXERCISE 2: OBJECT LIFECYCLE

Reinforces: object reachability and lifecycle, as just covered.

#	TITLE	FOCUS
1	Goal	Create objects, null a reference, optionally call System.gc(); describe reachability
2	Do this	Object → reference → null, then write observation notes
3	Key concept	An object becomes eligible for GC once no reachable reference points to it
4	Reference	exercises/exercise-02-lifecycle.md

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-lifecycle.md.

GARBAGE COLLECTION TRIGGERS

Garbage Collection in the JVM runs automatically when certain conditions occur to reclaim memory and maintain performance.

EXAMPLE SCENARIO

How a real application progresses from routine allocation to a possible OutOfMemoryError.

- Let the JVM manage GC — avoid calling System.gc() in production.
- Monitor memory usage and GC activity.
- Tune heap sizes and GC settings based on workload.
- Write memory-efficient code to reduce unnecessary object creation.
- GC does not necessarily run immediately when an object becomes unreachable.

KEY TAKEAWAY GC is about reclaiming unused memory at the right time to keep your application fast and stable.

TRY IT YOURSELF — EXERCISE 3: GARBAGE COLLECTION IN ACTION

Reinforces: the GC triggers you just saw.

#	TITLE	FOCUS
1	Goal	Allocate in a loop; run with a small heap; capture evidence that GC activity occurred
2	Command	java -Xmx64m YourApp
3	Key concept	A small -Xmx forces GC to run sooner and more often — makes it observable
4	Reference	exercises/exercise-03-gc-observe.md

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-gc-observe.md.

G1 GARBAGE COLLECTOR

Garbage-First, Region-Based, Low-Pause Garbage Collector in the JVM

1. HEAP STRUCTURE – REGION BASED

The heap is divided into fixed-size regions (1MB – 32MB).
Regions can be of different types.

2. HOW G1 WORKS

Allocation
New objects are allocated in Eden regions.

↓

Region Monitoring
G1 continuously monitors region usage and objects.

↓

Garbage-First Selection
Selects regions with most garbage to reclaim (best cost-benefit).

↓

Evacuation (Copying)
Live objects are copied to other regions.

↓

Update References
References are updated to point to new locations.

3. PAUSE TIME GOAL

G1 aims to keep pause times predictable and within the user-defined goal.

Pause Time Goal
e.g., -XX:MaxGCPauseMillis=200

4. CONCURRENT PHASES

Most of the work is done concurrently with application threads.

- **Concurrent Marking** — Marks live objects in the background.
- **Concurrent Refinement** — Refines the discovered references.
- **Concurrent Cleanup** — Cleans up regions after collection.

5. G1 COLLECTION CYCLE (TYPICAL FLOW)

Young GC (Minor GC) — Collect Eden regions.
→ **Concurrent Marking** — Find live objects in the background.
→ **Remark** — Pause to finalize marking.
→ **Cleanup** — Identify eligible regions for collection.
→ **Mixed GC** — Collect selected old regions + young regions.
→ **Concurrent Cleanup** — Finalize and prepare for next cycle.

Cycle repeats continuously

6. KEY FEATURES

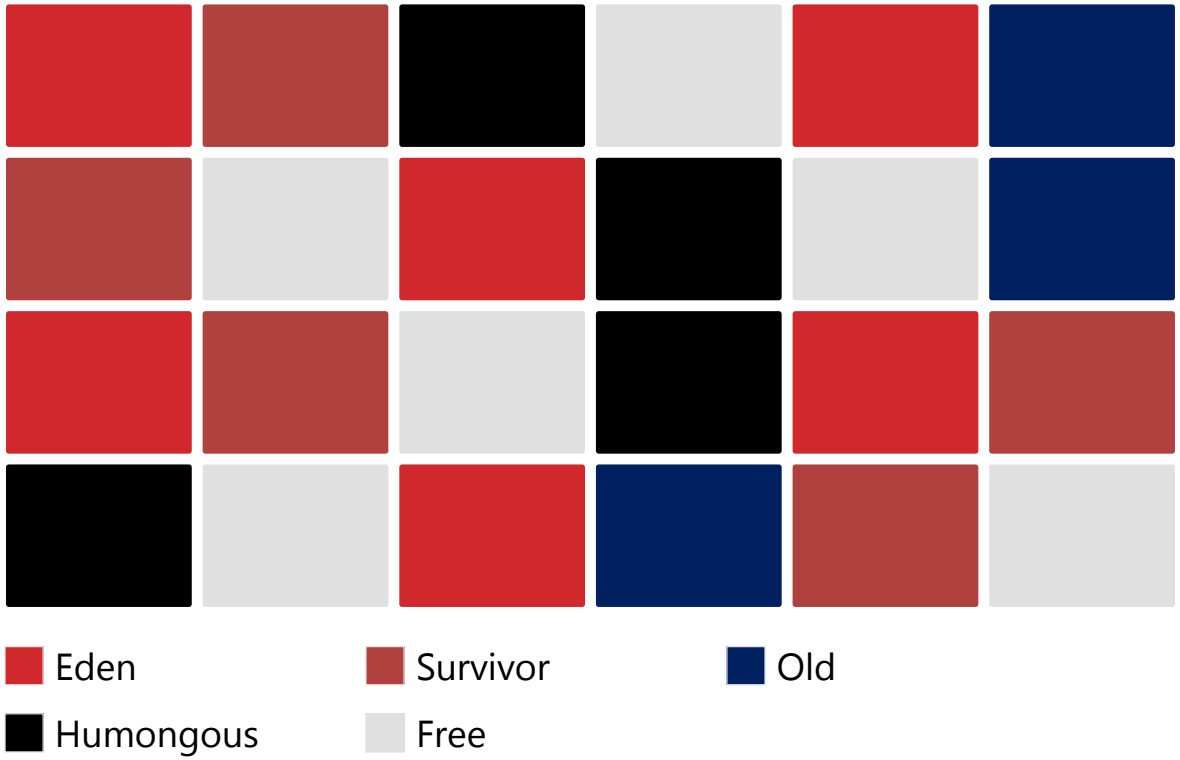
- Region-based memory management
- Predictable low pause times
- Garbage-first collection strategy
- Scales well for large heaps (GBs – TBs)
- Handles humongous objects efficiently

G1 is the default garbage collector in modern Java (Java 9+). It is designed for applications that require high throughput with predictable pause times.

G1 HEAP LAYOUT & GC CYCLE

Regions of equal size adapt roles over time; most of the cycle runs concurrently with your app.

HEAP LAYOUT (REGION BASED)



G1 GC CYCLE (HIGH LEVEL)

1. Young GC — collects young generation
2. Concurrent Marking — marks live objects
3. Remark — short STW pause to finish marking
4. Cleanup — prepares regions for evacuation
5. Mixed GC — collects young + selected old regions

KEY TAKEAWAY Only phases 1, 3, and Mixed GC pause the application — marking and cleanup run concurrently.

G1 — IMPORTANT JVM OPTIONS

The flags you'll actually reach for when tuning G1.

OPTION	PURPOSE
`-XX:+UseG1GC`	Enable G1 GC (default from JDK 9+)
`-XX:MaxGCPauseMillis=<ms>`	Set target maximum pause time (e.g., 200 ms)
`-XX:InitiatingHeapOccupancyPercent`	Start concurrent marking at n% heap usage (~45% default)
`-Xms / -Xmx`	Set initial and maximum heap size
`-Xlog:gc* (JDK 9+)`	Unified GC logging
Best Suited For	Large heaps (GBs–TBs), predictable pauses, fluctuating workloads, multi-core systems — and the JDK 9+ default.
Trade-offs	Slightly higher CPU/memory use than simpler collectors; best results still require tuning.

KEY TAKEAWAY Monitor GC logs and metrics, and set realistic pause-time goals — too low a target reduces throughput.

TRY IT YOURSELF — EXERCISE 4: G1 COLLECTOR FLAG

Reinforces: the G1 collector you just saw.

#	TITLE	FOCUS
1	Goal	Run a short program with -XX:+UseG1GC and record the command
2	Command	java -XX:+UseG1GC YourApp
3	Key concept	G1 is the JVM's default collector since Java 9 — this flag makes the choice explicit
4	Reference	exercises/exercise-04-g1.md

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-g1.md.

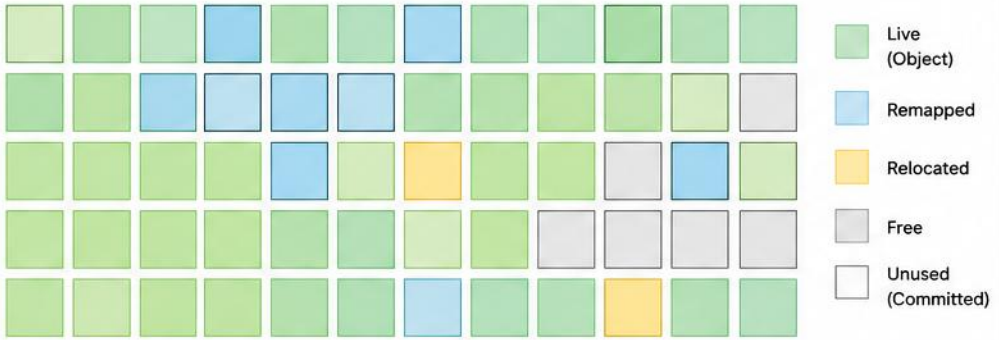
ZGC (Z Garbage Collector) – Java

Ultra-low pause, scalable garbage collector for large heaps (TB scale)

Key Characteristics

- **Ultra-Low Pause Times**
  Pauses < 10ms (typically 1–2ms)
- **Scales to Multi-TB Heaps**
  Designed for very large memory
- **Concurrent & Parallel**
  Most of the work is done concurrently with the application
- **Load Barriers**
  Ensures memory consistency with colored pointers
- **High Throughput**
  Low pause with strong application throughput

ZGC Heap (Region Based)



ZGC Execution Phases

- **Mark** — Concurrent — Identify live objects
- **Relocate** — Concurrent — Copy live objects to new regions
- **Remap** — Mostly Concurrent (very short pause) — Update references using load barriers
- **Reclaim** — Concurrent — Free old regions
- **Repeat** — Continuous cycle with minimal pauses

How ZGC Works

1. **Mark**
   Concurrent marking identifies live objects.
2. **Relocate**
   Live objects are copied to new regions concurrently.
3. **Remap**
   Pointers to moved objects are updated using load barriers.
4. **Reclaim**
   Old regions are freed and returned to the free list.
5. **Repeat**
   Process runs continuously with very short pauses.

Load Barriers (Colored Pointers)

Used by ZGC to keep track of object location state

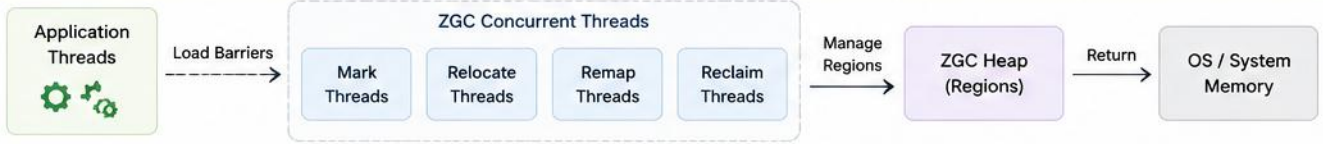
Colors represent pointer state

00	Finalizable (Normal)
01	Remapped (Needs remap)
10	Marked
11	Weakly reachable

Benefits

- **Pauses < 10ms**
  Consistent low latency
- **Scales to TBs**
  Handles very large heaps efficiently
- **High Throughput**
  Designed for production workloads
- **Stable & Predictable**
  Low latency with high reliability

ZGC Architecture



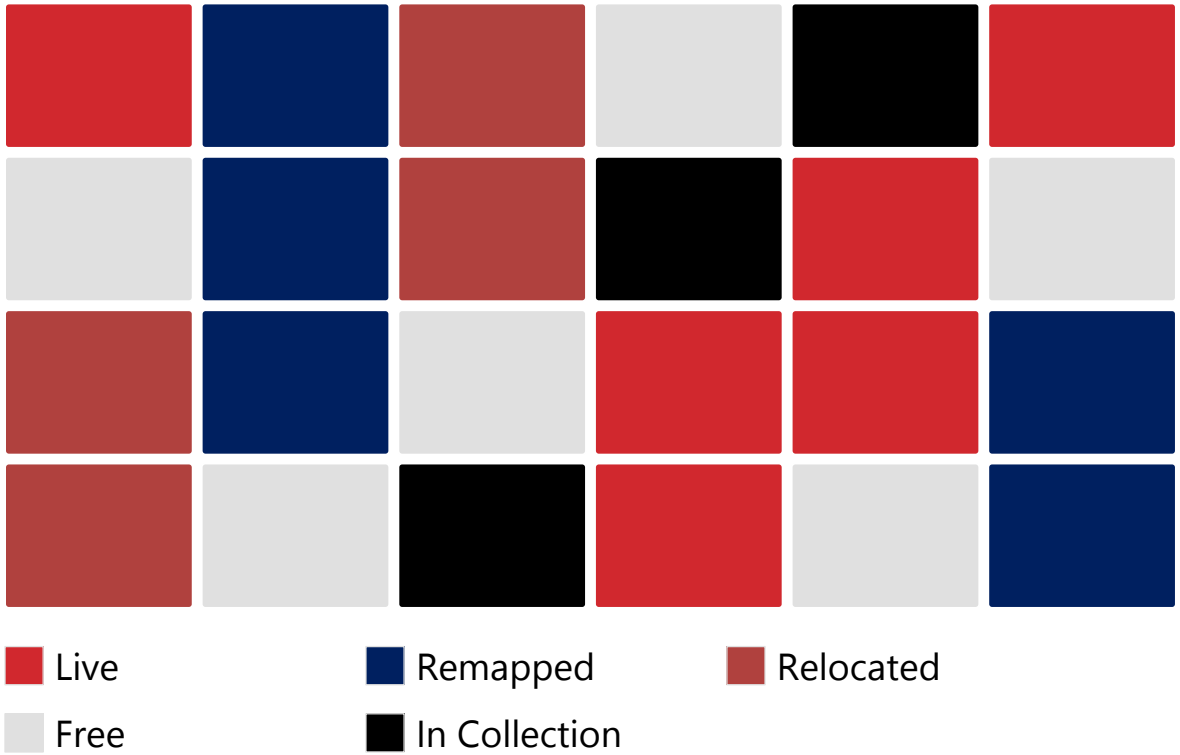
Use Cases

- Large heap applications (100GB – Multi-TB)
- Low latency services
- Microservices, Real-time analytics, Trading systems
- Cloud-native and containerized workloads

ZGC HEAP LAYOUT & CYCLE

ZGC continuously relocates live objects and remaps pointers, working in small steps to keep pauses minimal.

HEAP LAYOUT (REGION BASED)



ZGC CYCLE (HIGH LEVEL)

1. Mark — concurrent marking to identify live objects
2. Relocate — live objects moved to new locations
3. Remap — pointers remapped to new locations
4. Reclaim — old regions with no live objects reclaimed

Repeat continuously — no full stop-the-world phase.

KEY TAKEAWAY Pause times are mostly under 1 ms, even for very large heaps — ZGC never stops the world for a full collection.

ZGC — IMPORTANT JVM OPTIONS

The flags you'll actually reach for when tuning ZGC.

OPTION	PURPOSE
`-XX:+UseZGC`	Enable ZGC
`-Xms<size> -Xmx<size>`	Set initial and maximum heap size
`-XX:ZCollectionInterval=<ms>`	(Optional) Soft limit for GC interval
`-XX:ZUncommitDelay=<s>`	Delay before returning memory to the OS
`-Xlog:gc*,gc+z*=debug`	Detailed ZGC logging
Best Suited For	Ultra-low latency needs (<1ms pauses), very large heaps, real-time and high-throughput services.
Trade-offs	Higher CPU overhead and more metadata memory than G1; not optimized for small heaps.

KEY TAKEAWAY Always test and tune ZGC with your real workload — use the latest JDK for ongoing improvements.

TRY IT YOURSELF — EXERCISE 5: ZGC FLAG

Reinforces: the ZGC collector you just saw.

#	TITLE	FOCUS
1	Goal	Select ZGC explicitly and compare its log against Exercise 4's G1 run
2	Command	java -XX:+UseZGC -Xlog:gc GcObserve
3	Key concept	ZGC does most of its work concurrently — pauses stay tiny even on large heaps
4	Reference	exercises/exercise-05-zgc.md

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-zgc.md.

MEMORY LEAKS EXPLAINED

Memory that is no longer needed by the application is not released and becomes unreachable for reuse.

What Is a Memory Leak?

- Objects no longer in use are still referenced.
- This prevents the GC from reclaiming memory.

Why It Matters

- Increased memory usage over time
- Degraded application performance
- OutOfMemoryError (OOM) crashes

How Java Manages Memory

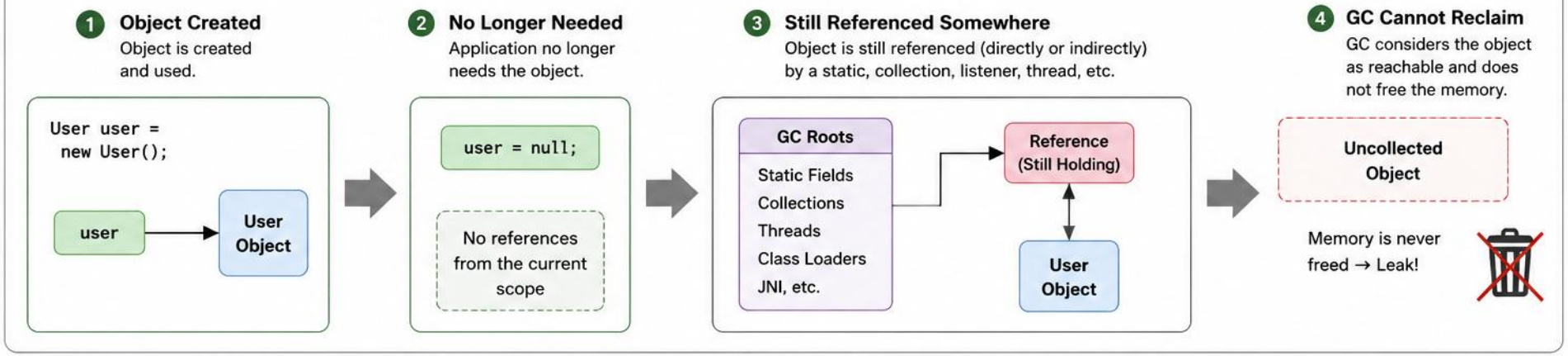
- Objects are created in Heap memory.
- Unreachable objects are collected by GC.
- If still reachable, memory is not freed.

KEY TAKEAWAY A memory leak in Java isn't lost memory — it's memory that's still referenced when it shouldn't be.

MEMORY LEAKS IN JAVA

A memory leak occurs when an application unintentionally retains objects that are no longer needed, preventing Garbage Collector (GC) from reclaiming memory.

HOW MEMORY LEAKS HAPPEN



COMMON CAUSES OF MEMORY LEAKS

Static Collections
Objects added to static collections are never released.

Unclosed Resources
Streams, connections, sockets not closed properly.

Listeners / Callbacks
Listeners registered but not unregistered.

Inner Classes
Non-static inner classes hold implicit reference to outer class.

Thread Local Variables
ThreadLocal values not removed after use.

PREVENTION & DETECTION

BEST PRACTICES

- Clear collections when no longer needed.
- Close resources in finally block or try-with-resources.
- Unregister listeners and callbacks.
- Use WeakReference / SoftReference where appropriate.
- Remove ThreadLocal values using ThreadLocal.remove().

DETECTION TOOLS

VisualVM
Monitor heap usage, detect leaks.

Eclipse MAT
Analyze heap dump to find leaks.

JProfiler / YourKit
Identify memory retention paths.

GC Logs (-Xlog:gc*)
Analyze GC behavior and memory usage.

Memory leaks increase memory usage over time, degrade performance, and can eventually lead to OutOfMemoryError.

HOW MEMORY LEAKS HAPPEN

Five of the most common causes — all involving cleanup that never happens.

Unintentional References

- Objects still referenced by variables, static fields, or collections.

Static Collections

- Objects added to static collections are never removed.

Cache Without Expiration

- Caches that grow continuously without removing old entries.

Unremoved Listeners / Callbacks

- Listeners hold references even after they're no longer needed.

Resource Not Closed

- Open files, streams, sockets, and DB connections keep resources.

```
public class LeakyService {  // static list holds refs forever
    private static List<User> users = new ArrayList<>();
    public void process(User user) {
        users.add(user);   // never removed
    }
}
```

KEY TAKEAWAY Every listener you register and every resource you open is a cleanup obligation.

DETECT, DIAGNOSE, FIX & VALIDATE

How to find leaks before production, and how to avoid writing them in the first place.

How to Detect

- Monitor heap usage over time (VisualVM, JConsole).
- Take thread dumps and heap dumps.
- Analyze heap dumps in Eclipse MAT or VisualVM.
- Check GC logs for increasing heap after GC.

How to Prevent

- Remove references when objects are no longer needed.
- Use WeakHashMap and WeakReference for caches.
- Clear collections and caches periodically.
- Always close resources — try-with-resources.
- Unregister listeners, callbacks, and observers.

KEY TAKEAWAY If memory keeps growing without reason, you probably have a memory leak — keep heap usage stable over time.

TRY IT YOURSELF — EXERCISE 6: LEAK SKETCH (SAFE)

Reinforces: memory leaks and GC roots, as just covered.

#	TITLE	FOCUS
1	Goal	Show a growing static List that retains objects (cap the size — do not OOM)
2	Do this	Stop at e.g. 10,000 entries, then write a root-cause note
3	Key concept	A static field is a GC root — objects it references are never eligible for collection
4	Reference	exercises/exercise-06-leak-sketch.md

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-leak-sketch.md.

DIAGNOSING MEMORY ISSUES

A systematic approach and the right tools help you find the root cause quickly.

Common Symptoms

- High memory usage over time
- Frequent or long GC pauses
- OutOfMemoryError (OOM)
- Slow application / degraded throughput

Goal

- Identify what is consuming memory,
- why it is growing, and how to fix it.

Key Principle

- Measure → Analyze →
- Identify Root Cause → Fix → Validate

KEY TAKEAWAY The JVM manages memory, but your code determines what becomes unreachable.

DIAGNOSING MEMORY ISSUES IN JAVA

Identify the problem → Capture evidence → Analyze → Fix & Validate

1. IDENTIFY SYMPTOMS

- High memory usage
- OutOfMemoryError
- Frequent GC / long GC pauses
- Slow performance

2. CAPTURE EVIDENCE

- Heap Dump (.hprof)
- Thread Dump
- GC Logs
- JVM / OS metrics

3. ANALYZE

- Find what's using memory
- Find leak / retention path
- Check GC behavior
- Correlate with metrics

4. FIX & VALIDATE

- Fix memory leak / issue
- Optimize code/config
- Re-run & monitor
- Validate improvement

HOW TO CAPTURE

HEAP DUMP

.hprof

```
jcmd <pid> GC.heap_dump
/path/dump.hprof
```

Captures a snapshot of heap objects.

THREAD DUMP

```
jcmd <pid> Thread.print
(or) Ctrl+\ in jstack
```

Shows all threads and their stack traces.

GC LOGS

gc.log

```
-Xlog:gc*:file=gc.log:time,uptime,
level,tags (JDK 11+)
```

Shows GC activity, pauses, heap usage over time.

METRICS

```
jstat -gcutil <pid> 5s
(or) JMX / Prometheus
```

Real-time stats: heap usage, GC count, pause time, etc.

OS METRICS

```
top / htop / free -m
vmstat / iostat
```

Check system memory, swap, CPU, paging.

ANALYZE WITH TOOLS

ECLIPSE MAT

MAT

- Open .hprof
- Find Dominator Tree
- Top Consumers
- Path To GC Roots
- Leak Suspects Report

VISUALVM

- Monitor memory/CPU
- Take Heap Dump
- Thread Dump
- Sampler / Profiler
- GC visualization

JMC (Java Mission Control)

- Flight Recorder data
- Memory leak profile
- Allocation in new TLAB
- GC pauses & trends
- Export reports

GC LOG ANALYZERS

GC

- GCeasy.io
- GCViewer
- Understand GC pauses
- Heap growth trends
- Tuning suggestions

JFR (Java Flight Recorder)

- Low overhead profiling
- Allocation profiling
- Leak detection
- Events & timelines

COMMON MEMORY ISSUES & WHAT TO CHECK

ISSUE	WHAT TO CHECK
Memory Leak	Large heap growth, many live objects → Use Heap Dump, Dominator Tree, Path To GC Roots
High GC Overhead	High GC time %, frequent GC → Check GC logs, heap size, object churn, tuning
Excessive Object Creation	High allocation rate → Use JFR/VisualVM allocation profiling, optimize code
Fragmentation / Large Objects	Many Humongous objects → Check GC logs, object size, consider G1/ZGC tuning

BEST PRACTICES

- ✓ Enable GC logging in all environments
- ✓ Monitor heap usage and GC pauses continuously
- ✓ Take heap dumps before system crash (if possible)
- ✓ Reproduce issue in lower environment
- ✓ Fix the root cause, not just increase heap size
- ✓ Validate the fix with load and monitoring

STEPS 1-2 — OBSERVE, THEN ANALYZE

Before you can fix a memory problem, gather evidence — then turn it into understanding.

Monitor Metrics

- Heap usage, GC activity, system metrics.
- Tools: VisualVM, JConsole, Prometheus

Check GC Logs

- Heap usage, GC frequency, pause times.
- -Xlog:gc* or -XX:+PrintGCDetails

Look for Errors & Reproduce

- Check logs for OutOfMemoryError.
- Reproduce in a lower environment.

Heap Dumps

- Capture at the time of the issue.
- Analyze with Eclipse MAT, VisualVM

Live Profiling

- Top memory consumers.
- Object allocation rate.

Key Things to Look For

- Large objects / big collections.
- ClassLoader leaks, unbounded caches.

KEY TAKEAWAY You can't fix what you haven't measured — a heap dump is a snapshot, so capture it close to the moment of failure.

STEP 3 — IDENTIFY ROOT CAUSE

Five categories cover almost every memory problem you'll encounter.

Unintended Retention

Static collections, caches, singletons, listeners.

Unbounded Growth

Collections, maps, queues, or caches grow without limits.

Resource Leaks

Files, sockets, DB connections, streams not closed.

Incorrect Cache Usage

No eviction policy, or extremely large caches.

Poor Lifecycle Mgmt.

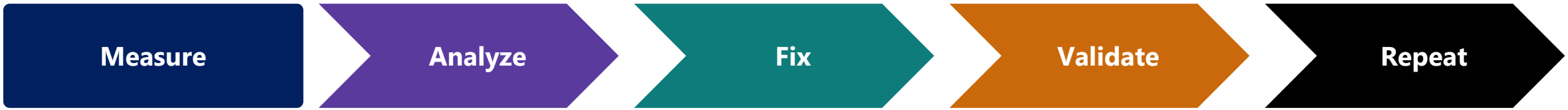
Objects kept longer than necessary.

KEY TAKEAWAY Root cause almost always comes down to something being kept alive longer than it needs to be.

STEP 4 — FIX, VALIDATE & REPEAT

Diagnosis is only half the job — closing the loop is what makes the fix stick.

- Remove unnecessary references; use appropriate data structures and limits.
- Close resources with finally blocks or try-with-resources; implement cache eviction policies.
- Optimize object lifecycle and clean up listeners, threads, etc.; redeploy and monitor to verify.



KEY TAKEAWAY Diagnosing memory issues is a continuous improvement cycle, not a one-time fix.

USEFUL JVM OPTIONS & POPULAR DIAGNOSTIC TOOLS

The flags and the tools most Java teams reach for when investigating a memory issue.

JVM OPTIONS

OPTION	PURPOSE
`-Xms / -Xmx`	Initial and maximum heap size
`-XX:+HeapDumpOnOutOfMemoryError`	Generate heap dump on OOM
`-XX:HeapDumpPath=<path>`	Heap dump file location
`-Xlog:gc* (JDK 9+)`	Unified GC logging
`-XX:NativeMemoryTracking`	Track native memory usage

POPULAR TOOLS

TOOL	PURPOSE
`VisualVM`	Monitor, profile, heap dumps
`Eclipse MAT`	Analyze heap dumps
`JConsole`	JMX monitoring
`JProfiler / YourKit`	Advanced profiling
`JFR (Flight Recorder)`	Low-overhead production profiling

KEY TAKEAWAY Configure heap-dump-on-OOM in every production environment — you only get one chance to capture it.

JAVA PERFORMANCE FUNDAMENTALS

Write efficient code. Use resources wisely. Understand what the JVM is doing.

CORE PRINCIPLES

Performance is a Feature
Fast apps deliver better UX and lower costs.

Measure, Don't Guess
Use metrics and profiling to find real bottlenecks.

Optimize Wisely
Avoid premature optimization; focus on high-impact changes.

Understand Trade-offs
Balance CPU, memory, latency and throughput.

Know Your Workload
Different applications need different tuning approaches.

1. KNOW YOUR BOTTLENECK

CPU Bound
High CPU, low I/O wait — check algorithms & locks.

Memory Bound
High GC activity, frequent pauses, OOM risk.

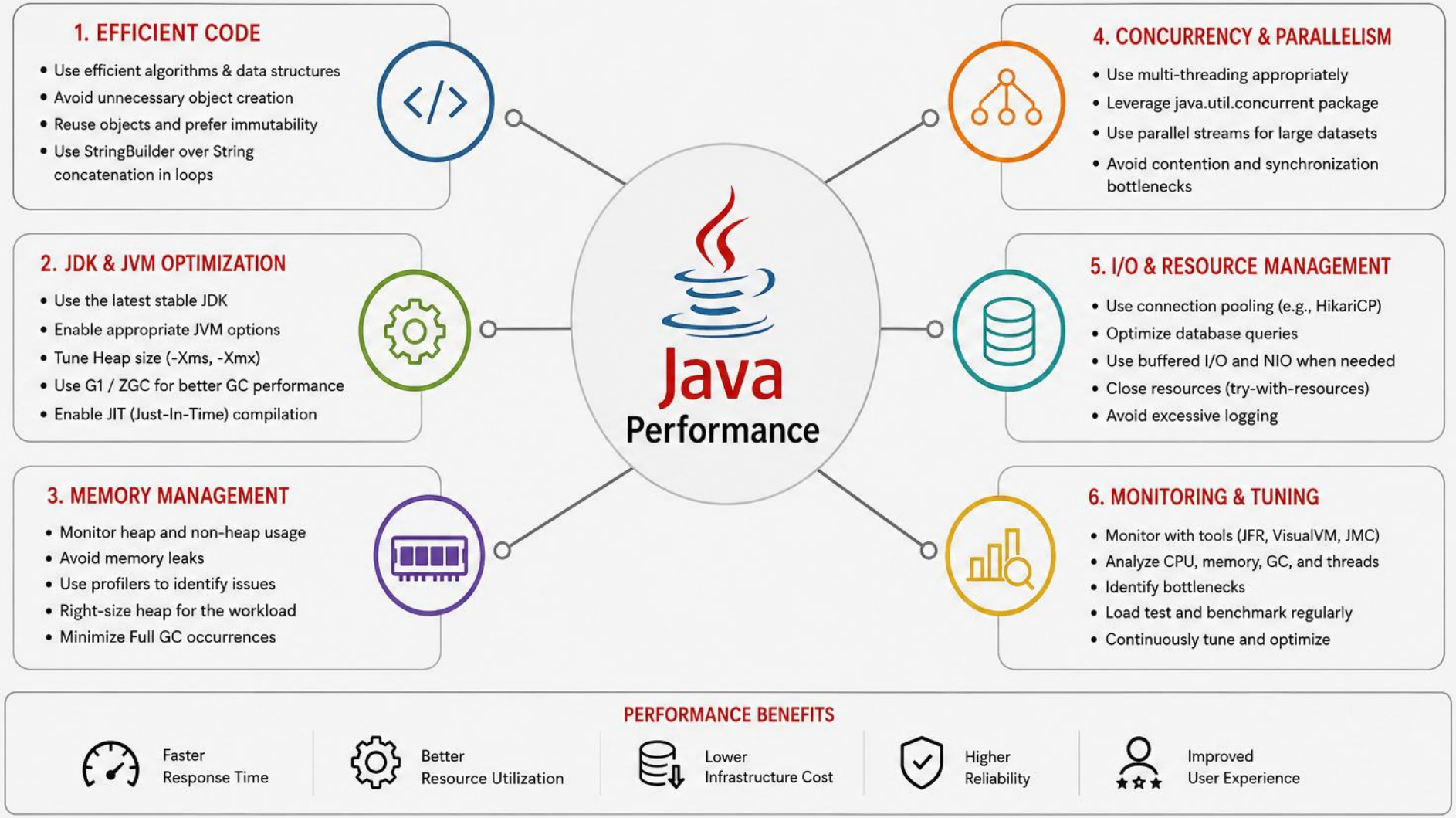
I/O Bound
High wait on disk, network, or database.

Lock Bound
High blocked time, low CPU — check contention.

KEY TAKEAWAY Performance work always starts by understanding which resource — CPU, memory, I/O, or time — is the constraint.

JAVA PERFORMANCE

Java performance depends on efficient code, optimized JVM behavior, proper resource management, and right-sizing the infrastructure.



2. WRITE EFFICIENT CODE

Small, everyday choices that compound into real performance gains.

Choose Right Data Structures

- ArrayList over Vector, HashMap over Hashtable
- StringBuilder over + for strings

Avoid Unnecessary Object Creation

- Reuse objects where possible.

Use Primitives, Not Wrappers

- Use int instead of Integer to reduce overhead.

Avoid Excessive Synchronization

- Use concurrent collections.
- Lock only when needed.

Minimize Method Call Overhead

- Keep hot-path methods small.
- Mark final where appropriate.

Prefer Enhanced For-Loop

- Cleaner and potentially optimized.

KEY TAKEAWAY Synchronization and method-call overhead matter most exactly where you can least afford them: hot paths.

3. MEMORY-EFFICIENT JAVA PRACTICES

Concrete habits that keep allocations low and heap usage predictable.

Reduce Object Allocations

- Reuse objects.
- Use object pools only when measurements justify it.
- Be careful with autoboxing.

Bound Collections & Caches

- Set a max size or eviction policy on every cache.
- Avoid unbounded growth in maps, lists, and queues.
- Prefer a bounded LRU cache over a plain HashMap.

Right-Size the Heap

- Too small → GC pressure, OOM.
- Too large → longer GC pauses.
- Adjust -Xms/-Xmx only after measuring, not by guessing.

KEY TAKEAWAY Most performance problems are caused by memory misuse and unnecessary allocations.

4. AVOID COMMON PERFORMANCE ANTI-PATTERNS

Six habits that quietly degrade an otherwise well-designed application.

Premature Optimization

Optimizing before measuring wastes time.

Busy Waiting / Spin Loops

Consumes CPU without doing useful work.

Blocking on Critical Paths

Avoid blocking I/O or DB calls in latency-sensitive code.

Large Synchronized Blocks

Reduces concurrency and throughput.

Unbounded Collections

Collections that grow without limits cause memory issues.

Logging in Hot Paths

Excessive logging slows down the application.

KEY TAKEAWAY Every one of these anti-patterns is easy to introduce accidentally and easy to avoid once you know to look for it.

5. MEASURE & PROFILE

You can't optimize what you don't measure.

📊 Key Metrics

- Response Time / Latency
- Throughput
- CPU Usage
- Memory Usage
- GC Activity
- Error Rate

🛠️ Tools

- JMH — Microbenchmarking
- VisualVM — Profiling & Monitoring
- Java Flight Recorder (JFR)
- JConsole, GC Logs, YourKit / JProfiler

KEY TAKEAWAY JMH, VisualVM, and JFR cover almost every measurement need a Java developer will have.

6. UNDERSTAND THE JVM

What the JVM already optimizes for you, and how to stay out of its way.

⚙ JVM Optimizations

- JIT (Just-In-Time) Compilation
- Escape Analysis
- Inline Caching
- Dead Code Elimination
- Loop Unrolling

✅ What You Can Do

- Write simple, clean code.
- Warm up your application.
- Avoid reflection in hot paths.
- Keep code JIT-friendly.
- Let the JVM do its job.

KEY TAKEAWAY The JVM's JIT compiler does an enormous amount of optimization automatically — clean, simple code helps it help you.

7. EXAMPLE: STRING CONCATENATION

String concatenation in a loop creates many temporary objects — StringBuilder avoids that entirely.

INEFFICIENT

```
String result = "";
for (String s : list) {
    result += s;      // Creates many objects
}
```

EFFICIENT

```
StringBuilder sb = new StringBuilder();
for (String s : list) {
    sb.append(s);
}
String result = sb.toString();
```

- Measure before you optimize; profile in a production-like environment.
- Focus on the bottlenecks, not the whole code; test with real workloads.

KEY TAKEAWAY Use StringBuilder (or StringBuffer) instead of + concatenation inside loops.

TRY IT YOURSELF — EXERCISE 7: STRING VS STRINGBUILDER

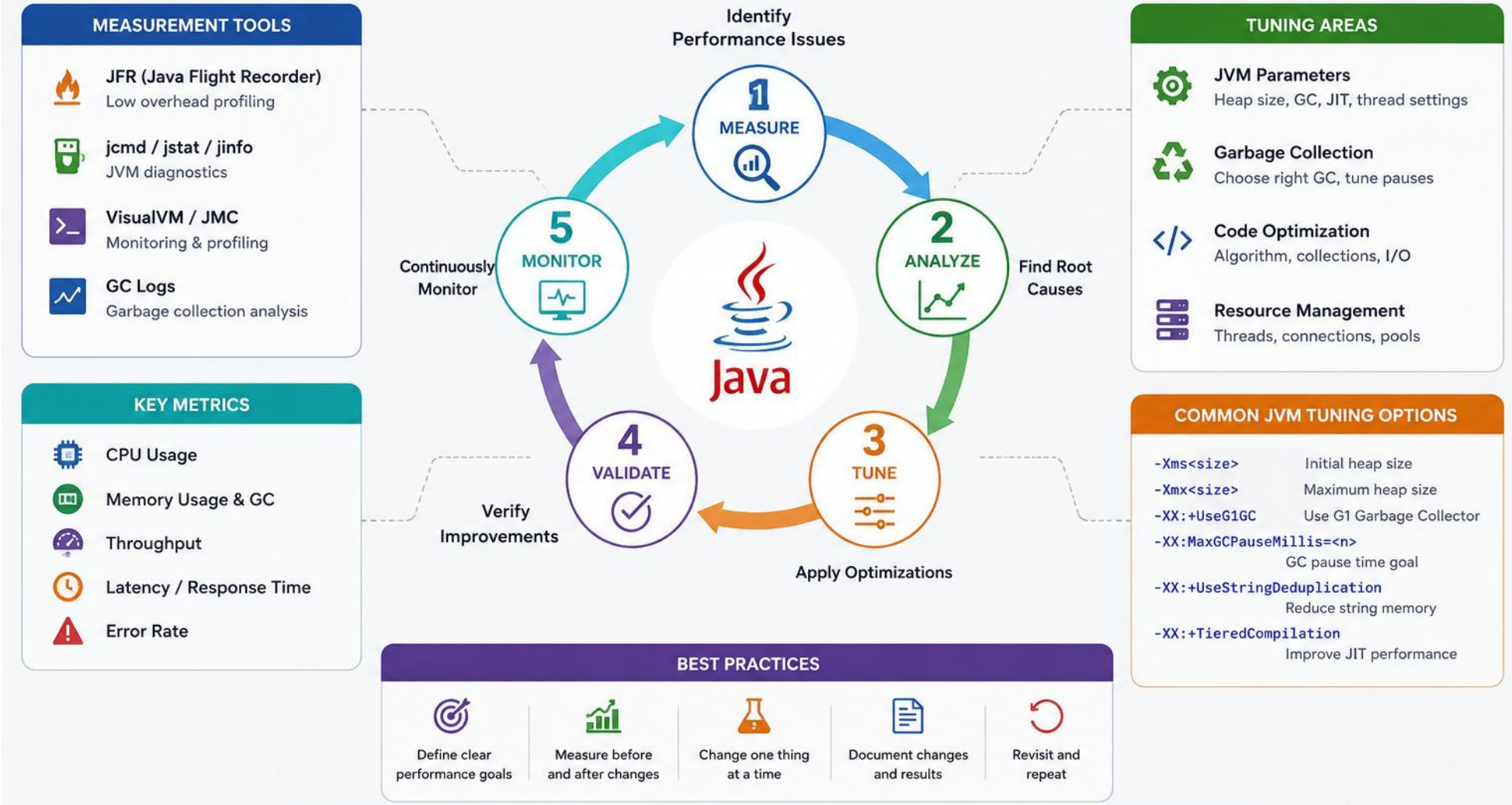
Reinforces: the string concatenation example you just saw.

#	TITLE	FOCUS
1	Goal	Compare naive String concat in a loop vs. StringBuilder for N ≈ 50,000 (timing)
2	Do this	Wrap each loop in System.nanoTime() calls and print the duration
3	Key concept	Each + on a String creates a new object; StringBuilder mutates one buffer instead
4	Reference	exercises/exercise-07-string-vs-builder.md

TRY IT NOW Complete Exercise 7 before continuing — see exercises/exercise-07-string-vs-builder.md.

JAVA PERFORMANCE TUNING

A Continuous Process for Optimal Performance



2-3. KEY AREAS TO TUNE & JVM BASICS

Five categories cover most real-world tuning work; heap sizing is the single biggest lever.

Code & Algorithms
Efficient algorithms; avoid unnecessary work.

Memory & GC
Right-size the heap; reduce allocations.

Data Structures
Right collection; set initial capacity.

Concurrency
Minimize locks; tune thread pools.

I/O & Resources
Buffered streams, connection pooling.

HEAP SIZING — set -Xms/-Xmx properly. Too small → frequent GC. Too large → long GC pauses.

GC	BEST FOR	CHARACTERISTICS
`G1 (default)`	Most applications	Balanced throughput and pause times
`ZGC`	Very large heaps, low latency	Sub-millisecond pauses
`Shenandoah`	Large heaps, low latency	Concurrent, low pauses
`Serial / Parallel`	Small heaps, batch jobs	Throughput focused

KEY TAKEAWAY Choosing the right GC for your workload is a bigger lever than almost any other single JVM tuning flag.

4. MEASURE & ANALYZE

The essential toolset for turning tuning into a data-driven exercise.

Essential Tools

- JDK Tools: jcmd, jstat, jmap, jstack, jinfo
- VisualVM: monitor, profile, heap dump
- JFR: low-overhead profiling
- OS Tools: top, vmstat, iostat, netstat

What to Monitor

- Throughput — requests/sec, transactions/sec
- Latency — average, P95, P99
- CPU & memory usage
- I/O, database, network dependencies

KEY TAKEAWAY Ask the right question for each metric: is throughput meeting goals? Are response times acceptable?

5. TUNING WORKFLOW

The same five-step loop, every time you tune.

KEY TAKEAWAY The best tuning is the one that solves the real bottleneck with the least complexity.

6. QUICK TUNING CHECKLIST

Eight items worth reviewing on every performance pass.

✅ Heap & GC

- Right-size heap (-Xms and -Xmx)
- Use the right GC for your workload

✅ Code & Data

- Review and optimize hot code paths
- Reduce object creation and large objects
- Choose appropriate data structures

✅ Concurrency & I/O

- Tune thread pools and concurrency
- Use connection pooling and caching
- Monitor, analyze, iterate

KEY TAKEAWAY Get it working correctly first, then tune — always in a production-like environment with real data.

7. GOOD PRACTICES

Closing guidance to keep tuning disciplined and repeatable.

- Get it working correctly first, then tune.
- Profile in an environment similar to production, with production-like data and load.
- Document every change and its impact.
- Avoid premature optimization; continuously monitor after deploying changes.

GOLDEN RULE

Measure → Analyze → Change → Validate → Repeat

KEY TAKEAWAY Performance tuning is a continuous process — understand your app, measure accurately, change carefully, validate consistently.

COMMON MEMORY PROBLEMS IN JAVA

Memory issues can degrade performance, cause long GC pauses, or crash your application.

Detect Early

Monitor memory usage and GC activity.

Understand Root Cause

Find what is holding memory and why it's growing.

Fix the Cause

Change code or configuration to remove the issue.

Validate

Measure again to ensure the problem is resolved.

Prevent

Follow best practices to avoid recurring issues.

KEY TAKEAWAY JVM manages memory automatically, but your code decides what becomes unreachable.

MEMORY PROBLEMS IN JAVA

Common memory problems can degrade performance, cause application crashes, or lead to unstable behavior in production systems.

MEMORY PROBLEMS

1. OUT OF MEMORY ERROR

The JVM cannot allocate more memory for objects or arrays.

Symptoms

- java.lang.OutOfMemoryError
- Application crashes
- Heap space fully utilized
- High memory usage

Impact

- Application termination
- Service downtime
- Poor user experience

2. MEMORY LEAKS

Objects that are no longer needed are still referenced and never garbage collected.

Symptoms

- Gradual increase in memory usage
- Frequent GC cycles
- Long GC pauses
- Eventually OOM Error

Impact

- Performance degradation
- Increased GC overhead
- Potential application crash

3. MEMORY FRAGMENTATION

Free memory exists but is broken into small blocks that cannot satisfy requests.

Symptoms

- OOM Error despite free memory
- Large allocations fail
- Unstable memory usage

Impact

- Allocation failures
- Increased GC activity
- Reduced throughput

4. EXCESSIVE OBJECTS

Too many objects are created due to inefficient code or caching.

Symptoms

- High object count
- Frequent GC
- High heap usage
- CPU overhead

Impact

- Poor performance
- High GC overhead
- Increased response time

5. PERMGEN / METASPACE EXHAUSTION

Too many classes are loaded or class unloading is not happening.

Symptoms

- java.lang.OutOfMemoryError: Metaspace
- Class loading failures
- Full GC doesn't help

Impact

- Application instability
- Deployment failures
- Potential outage

HOW TO DETECT AND DIAGNOSE

jstat
Monitor GC and memory statistics

jmap
Heap dump and memory usage

jvisualvm
Visual monitoring and profiling

MAT (Eclipse)
Analyze heap dumps and find leaks

GC Logs
Analyze GC behavior and pauses

PROBLEM PATTERNS — PART 1

Memory Leak, Heap-Space OOM, and Metaspace OOM.

Memory Leak

Objects no longer needed are still referenced and can't be collected.

Fix: remove unnecessary references, deregister listeners, use cache eviction.

OOM — Heap Space

Heap is full and the JVM cannot allocate more memory.

```
List<Integer> list = new
ArrayList<>();
while (true) { list.add(1); }
```

Fix: reduce allocations, increase heap only if truly required.

OOM — Metaspace

JVM runs out of Metaspace, which stores class metadata.

Fix: avoid unnecessary dynamic class loading; raise -XX:MaxMetaspaceSize.

KEY TAKEAWAY Each of these problems has a distinct signature in GC logs and heap dumps once you know what to look for.

PROBLEM PATTERNS — PART 2

GC Thrashing, Large Object/Array Issues, and Unclosed Resources.

GC Thrashing

JVM spends too much time in GC trying to free memory.

Fix: increase heap, reduce object churn, tune GC settings.

Large Object / Array Issues

Very large objects consume a lot of contiguous memory.

```
byte[] data = new byte[500 *
1024 * 1024];
// 500 MB array
```

Fix: process large payloads in chunks instead of loading them all at once.

Unclosed Resources

Files, sockets, and DB connections not closed properly.

```
InputStream in = new
FileInputStream("f");
// not closed
```

Fix: process in chunks / use try-with-resources.

KEY TAKEAWAY Large-object and unclosed-resource issues are easy to reproduce in a load test — don't wait for production to find them.

BEST PRACTICES — CONTINUED

The remaining habits, plus what to always keep in mind.

- Monitor production systems continuously.
- Test under realistic load.
- Review and tune regularly.

TIP

Memory problems rarely have a single cause. Measure, analyze, and fix the real root cause for lasting results.

KEY TAKEAWAY Efficient code + proper configuration + monitoring = high performance and stable applications.

LAB OVERVIEW — MEMORY AND GARBAGE COLLECTION

The graded Lab 4: turn every diagram in this module into working Java code you run and observe yourself.

LAB OBJECTIVES

- Explain stack (per-thread frames) vs. heap (shared objects), and trace locals across nested calls.
- Allocate heap objects and use identityHashCode() to tell references apart from objects.
- Narrate the object lifecycle: create → use → share → drop references → GC-eligible.
- Allocate a large batch, null the reference, and observe used memory before and after GC.
- Monitor heap usage with Runtime.getRuntime() (total / free / used / max).
- Reproduce a reachable-collection memory leak, fix it, and compare strong vs. WeakReference behavior.
- Enable GC logging, tune -Xms/-Xmx, and measure allocation cost with System.nanoTime().

KEY TAKEAWAY Lab 4 is built around one scenario — a banking batch job with a memory problem — solved through nine small Java programs.

LAB ENVIRONMENT & WORKFLOW

What you need, and the twelve-step loop you'll follow through Lab 4.

LAB ENVIRONMENT

JDK Version

Java 21 (required)

IDE

IntelliJ IDEA Community (primary) / VS Code (optional)

OS

Windows / macOS / Linux

Tools

jstat, jconsole, VisualVM (all optional)

LAB WORKFLOW

1. Build — write demos → **2. Trigger — GC & leaks** → **3. Measure — read logs** → **4. Reflect — fill & submit**

KEY TAKEAWAY You will write and run nine Java programs against one banking-batch-job scenario, tuning heap size and GC logging as you go.

USEFUL COMMANDS

The commands Lab 4 asks you to run and capture evidence from.

```
javac *.java                              # Compile all Lab 4 demos
java -Xlog:gc GarbageCollectionDemo       # Run with GC logging
jstat -gc <PID> 1000 5                  # GC stats every second, 5 times
jmap -dump:live,format=b,file=heap.hprof <PID>  # Optional heap dump (delete after use)
-Xms128m -Xmx512m PerformanceTest        # Heap sizing for the performance test
```

- Compile once with javac *.java, then run each demo on its own.
- Save GC log and memory-report snippets under notes/ as you go.
- Never commit .hprof heap dumps — delete them after a quick look.

KEY TAKEAWAY These are the exact commands Steps 6, 7, 10, and 11 of the Lab 4 guide ask you to run.

LAB 4 — STEPS 1–3: SETUP, STACK & HEAP

Building the workspace, then the stack and heap demos.

1 Workspace & Shared Types

- Create the examples/Lab4-MemoryManagement workspace.
- Add Person.java and MemoryMonitor.java (heap reports + GC hint).
- Confirm JDK 21 with java -version and javac -version.

`File: Person.java, MemoryMonitor.java`

2 StackExample — Nested Frames

- Declare locals in main; call methodA → methodB → methodC.
- Print primitive values, references, and identityHashCode per frame.
- Confirm each frame disappears when its method returns.

`File: StackExample.java`

3 HeapExample — Objects on the Heap

- Print a memory report before allocation.
- Create several small objects (Student, Employee, Customer, Book).
- Print a memory report after allocation and compare.

`File: HeapExample.java`

KEY TAKEAWAY These first three steps build the workspace and prove references (stack) are not the same as objects (heap).

LAB 4 — STEPS 4–6: LIFECYCLE, GC & LEAKS

From reachability to garbage collection to a real memory leak.

4 ObjectLifecycle — Create → Null → Eligible

- Create a Person and alias it with a second reference.
- Set both references to null.
- Trigger GC and print before/after memory reports.

`File: ObjectLifecycle.java`

5 GarbageCollectionDemo + GC Logging

- Allocate ~100,000 objects, then null the array reference.
- Trigger GC and print an After GC memory report.
- Re-run with -Xlog:gc and save a short log snippet.

`File: GarbageCollectionDemo.java`

6 MemoryLeakDemo — leak vs. fix

- leak mode: add to a static list that is never cleared.
- fix mode: use a local list, clear it, null it, then trigger GC.
- Compare rising vs. recovered used-memory readings.

`File: MemoryLeakDemo.java`

KEY TAKEAWAY The leak/fix comparison is worth 20 of the 100 rubric marks — the single largest line item.

LAB 4 — STEPS 7–8: WEAK REFERENCES & PERFORMANCE

Comparing strong and weak retention, then measuring allocation cost.

7 WeakReferenceDemo

- Hold a strong Person reference and confirm it survives GC.
- Wrap another Person in a WeakReference.
- Null the strong local, trigger GC, and check weakReference.get().
- Document that System.gc() is a hint, not a guarantee.

`File: WeakReferenceDemo.java`

8 PerformanceTest, Tools & Reflection

- Time allocations from 10 to 1,000,000 objects and fill the results table.
- Optionally attach jstat, jconsole, or VisualVM to a running demo.
- Answer the reflection questions in notes/lab4-answers.md.

`File: PerformanceTest.java`

KEY TAKEAWAY Step 11 is optional tooling — Step 12's reflection notes are required and carry real rubric marks.

LAB 4 SUBMISSION CHECKLIST

What Lab 4 asks you to submit, checkpoint by checkpoint.

Lab 4 Deliverables

- Nine working demo classes, from Person to PerformanceTest.
- Memory/GC screenshots, a -Xlog:gc snippet, and the performance table.
- notes/lab4-answers.md with reflection answers.
- No .hprof heap dumps, secrets, or verbatim solution copies.

Implementation Checkpoints

- Checkpoint A: workspace, Person, and MemoryMonitor compile.
- Checkpoint B: Stack, Heap, and Lifecycle demos run correctly.
- Checkpoint C: GC, leak, weak-reference, and performance demos complete.
- Checkpoint D: no .hprof in Git; screenshots show no secrets.
- Mark each checkpoint Pass or Fail in your lab notes.

KEY TAKEAWAY Graders score the reachability narrative and the leak/fix comparison as heavily as whether the code compiles.

LAB 4 EVALUATION RUBRIC (100 MARKS)

How the real, graded Lab 4 is scored, and the ground rules for submission.

EVALUATION CRITERIA	WEIGHT
`Stack vs. Heap Demonstration`	15%
`Object Lifecycle & GC Demo`	25%
`JVM Memory Monitoring`	15%
`Memory Leak Creation & Resolution`	20%
`Performance, Analysis & Code Quality`	25%

- Never commit .hprof heap dumps — they may contain sensitive data.
- Delete class files and dumps during cleanup; keep .java sources and notes.

KEY TAKEAWAY The leak/fix comparison and memory monitoring carry the most marks combined — measure, don't guess.

MODULE 4 SUMMARY

Java manages memory automatically through the JVM, but understanding how it works helps you build reliable, efficient, scalable applications.

1. MEMORY FUNDAMENTALS

Stack

- Per-thread memory.
- Primitives, references, method frames.
- Fixed size and very fast.
- Automatically managed — no GC involvement.

Heap

- Shared by all threads.
- Stores all objects and arrays.
- Dynamically allocated, GC-managed.
- Objects live here even when referenced from the stack.

2. OBJECT LIFECYCLE & GARBAGE COLLECTION

- **New** → **In Use** → **Unreachable** → **Garbage Collected** — the four stages every object passes through.
- An object becomes unreachable once no live reference points to it — the GC then reclaims it on its own schedule, not instantly.
- GC benefits: prevents memory leaks (when code is correct), improves developer productivity, and enhances application stability.

KEY TAKEAWAY Key idea: objects are created on the heap and referenced from the stack.

PRODUCTION MEMORY TROUBLESHOOTING CHECKLIST

Collector choice, the diagnostic loop, and the five failure modes to watch for.

Collector Choice G1 (balanced, default) vs. ZGC (ultra-low latency, huge heaps) — full comparison and JVM flags are on the G1 and ZGC option slides earlier in this module.

Diagnostic Loop Observe → Analyze → Identify Root Cause → Fix, Validate & Repeat (see the dedicated diagnosis workflow slides) — tools: JConsole, VisualVM, JFR, Eclipse MAT.

Memory Leaks

Referenced objects that can't be collected.

OOM – Heap

Heap full; JVM cannot allocate.

OOM – Metaspace

Too many classes loaded.

GC Thrashing

Too much time spent in GC.

Unclosed Resources

Files/connections/streams leaks.

KEY TAKEAWAY Most production memory incidents trace back to one of these five patterns — diagnose before you scale the heap.

PERFORMANCE TUNING RECAP & KEY TAKEAWAYS

The tuning habits that matter, and the six ideas this module builds toward.

6. PERFORMANCE TUNING BASICS

- Right-Size the Heap: set -Xms and -Xmx appropriately.
- Reduce Allocations: reuse objects, use object pools.
- Use Efficient Data Structures: avoid boxing.
- Minimize Synchronization: use concurrent collections.
- I/O and Resources: buffered I/O, close resources promptly.
- Measure & Iterate: validate and repeat.

KEY TAKEAWAYS

- Understand how stack and heap work, and how objects live and die.
- GC reclaims memory automatically — your code determines what is reachable.
- Choose the appropriate GC and tune heap settings for your workload.
- Know common memory problems and detect them early.
- Use the right tools to measure, diagnose, and improve performance.
- Optimize iteratively — focus on real bottlenecks, not assumptions.

KEY TAKEAWAY Goal: write Java applications that use memory efficiently, perform well, and scale reliably in production.

KNOWLEDGE CHECK — MULTIPLE CHOICE (1–5)

Choose the best answer. Correct answers are marked ✓.

1. Which memory area in Java is used for local variables and method calls?

A) Heap **B) Stack ✓** C) Metaspace D) Code Cache

2. Which area stores all objects and arrays in Java?

A) Stack **B) Heap ✓** C) PC Register D) Native Method Stack

3. Which of the following is TRUE about object reachability?

A) Objects on the stack are never reachable. B) Only objects in the heap can be reachable. **C) Unreachable objects are candidates for GC. ✓** D) Null references are always reachable.

4. Which GC algorithm is the default in most modern JDK versions (11+)?

A) Serial GC B) Parallel GC **C) G1 GC ✓** D) ZGC

5. Which GC is designed for very low pause times for huge heaps (TBs)?

A) G1 GC **B) ZGC ✓** C) JGC D) Shenandoah

KEY TAKEAWAY Five more multiple-choice questions continue on the next slide.

KNOWLEDGE CHECK — MULTIPLE CHOICE (6–10)

Choose the best answer. Correct answers are marked ✓.

6. What is a common cause of OutOfMemoryError: Java heap space?

A) Too many threads **B) Heap is full and JVM cannot allocate more memory ✓** C) Stack overflow D) Too many open files

7. Which of the following is an example of a memory leak?

A) Object becomes unreachable after method returns **B) Static collection holding references to objects ✓** C) Short-lived objects created in a loop D) Objects collected during GC

8. What command can you use to capture a heap dump?

A) jcmd <pid> GC.heap_dump <file.hprof> ✓ B) jcmd <pid> GC.run C) jmap -dump <pid> D) jstat -gc <pid> 1000

9. Which tool is used to analyze heap dumps and find memory leaks?

A) JConsole B) VisualVM C) Eclipse MAT **D) All of the above ✓**

10. Which of the following helps reduce GC frequency?

A) Increase object churn **B) Use object pooling where appropriate ✓** C) Create unnecessary temporary objects D) Use large, short-lived collections

KEY TAKEAWAY 20 marks total across this knowledge check — 10 for multiple choice.

KNOWLEDGE CHECK — TRUE OR FALSE

Five statements, one point each.

STATEMENT	ANSWER
`Stack memory is shared among all threads.`	False
`The Garbage Collector reclaims memory of unreachable objects.`	True
`ZGC is not suitable for large heaps.`	False
`OOM: Metaspace occurs when class metadata grows beyond the Metaspace limit.`	True
`Tools like VisualVM can show CPU usage but not memory.`	False

KEY TAKEAWAY Stack memory is per-thread, not shared — that distinction trips up a lot of learners.

KNOWLEDGE CHECK — SHORT ANSWER (1–2)

Two points each.

1 Stack vs. Heap — Two Key Differences

- Stack stores local variables and method call frames; Heap stores objects and arrays.
- Stack is automatically managed and thread-specific; Heap is shared among threads and managed by the GC.

2 Two Ways Memory Leaks Occur

- Static collections retaining object references.
- Resources (files, sockets, DB connections) not closed properly.
- (Other valid answers: listeners/callbacks not deregistered, caches without eviction.)

KEY TAKEAWAY Both leak causes come down to the same root idea: something still holds a reference.

SHORT ANSWER (3) & SCENARIO (1)

G1's benefits, and choosing a GC under real symptoms.

Two Benefits of G1 GC

- Predictable pause times.
- Good balance between throughput and latency for large heap applications.

Scenario: 8GB Heap, Long GC Pauses, Short-Lived Objects

- a) G1 GC — optimized for large heaps, handles short-lived objects via generational collection, predictable pauses.
- b) Adjust heap sizing (-Xms/-Xmx); set -XX:MaxGCPauseMillis=<ms>.

KEY TAKEAWAY When symptoms point to long pauses on a large heap of short-lived objects, G1 is almost always the right default.

SCENARIO BASED (2)

OutOfMemoryError: Java heap space in production — what do you do?

List three steps to diagnose and fix the issue:

- i. Capture and analyze a heap dump using VisualVM or Eclipse MAT.
- ii. Review GC logs and monitor heap usage to identify leaks or excessive allocations.
- iii. Fix the root cause (remove memory leaks, reduce object creation, optimize data structures) and increase heap size if necessary.

KEY TAKEAWAY Diagnose before you scale — increasing heap size without fixing the root cause just delays the crash.

KNOWLEDGE CHECK — MATCH THE FOLLOWING

One point each. Answers are shown alongside each term.

TERM	ANSWER	DESCRIPTION
1. Heap Dump	E	Snapshot of heap memory for analysis
2. GC Throughput	C	Measure of how much time is spent doing useful work
3. Metaspace	A	Memory area that stores class metadata
4. Stop-The-World	D	Pauses all application threads during GC
5. Object Pooling	B	Technique to reuse objects to reduce allocations

KEY TAKEAWAY That's the full 20-mark Module 4 knowledge check — nicely done.

Nicely done

You can reason about memory, choose the right GC, and diagnose performance problems like a pro.

The Java Collections Framework is next in your toolkit.